

循环的魔力

for/while与嵌套循环 · 14题 · C++ & Python 双语对照

第三周，小鲁发现了一个令他震惊的事实："一个for循环能替我做100行重复代码？！"小华笑了："这就是**循环**的威力——编程从手动执行进化到自动重复的时刻，就像从步行变成了骑车。能识别出重复模式并用循环解决，才是真正学会了编程思维。"

本章14道题，小鲁将在小华的指导下，从最简单的for循环开始，逐步掌握while循环、嵌套循环和条件循环。更重要的是，学会**"什么时候用循环，什么时候用公式"**——循环是工具，不是信仰。当数学公式能一行搞定1000次循环时，数学和编程的结合才真正开始。

本章角色

-  **小鲁** (初学者) · 从"手动写100行"的困惑中觉醒，学习循环的力量
-  **小华** (算法高手) · ACM集训队队长，能用最生动的比喻解释while vs for
-  **小栋** (工程师) · 喜欢用实际问题挑战小鲁，代表真实编程场景
-  **小嘉** (校主精神) · 偶尔现身，用"自强不息"激励大家在算法之路上坚持

C++ vs Python 循环速查

- for循环**: `for(int i=0;i`
- 步长**: `i+=2` → `range(0,N,2)` (注意range的end不包含!)
- while循环**: `while(cond){...}` → `while cond:...` (几乎一样)
- 不定长输入**: `while(cin>>x)` → `while True: x=input(); if not x: break`
- 嵌套循环**: 两层for在两种语言中语法不同、逻辑相同
- Python独有**: 列表推导式一行替代循环+条件+赋值。生成器表达式处理大数据无内存压力

NQ029 偶数 AcWing 708

"我想打印出2到100之间的所有偶数，难道要写49行cout？！"小鲁被这重复劳动吓到了。小华指了指屏幕："看到这个模式了吗？2,4,6,8...100——每个数比前一个大2。这种重复操作正是循环的用武之地。一行for循环，49行的活一秒搞定——这就是为什么程序员永远不会被机器取代：因为我们知道什么时候让机器替我们干活。"

- 输入

无。
- 输出

每行一个偶数，从2到100。
- 范围

无

输入	输出
(无输入)	2
	4

6
...
100

思路 for循环的基础形态：设定起始值、终止条件、步长。C++: `for(int i=2;i<=100;i+=2)`, Python: `for i in range(2,101,2)`。注意range的end是不包含的，所以写101而非100。

技巧 Python的range(start, end, step)中end是exclusive。这是Python初学者最常见的off-by-one错误来源。i+=2比i=i+2更简洁——在所有类C语言中都通用。

C++

```
#include <iostream>

using namespace std;

int main()
{
    for (int i = 2; i <= 100; i += 2) cout << i << endl;

    return 0;
}
```

Python

```
for i in range(2, 101, 2):
    print(i)
```

NQ030 奇数 AcWing 709

"偶数搞定了，那奇数呢？"小鲁问。小华写下了：`for i in range(1, X+1, 2)`。"你看，只需改两个数——起始值从2改成1，步长保持2。奇偶的区别只差一个起始位置。"小鲁恍然大悟："所以循环的威力就在于——一个模式能生成无穷多个结果。从1到X的所有奇数，只需要一行range！"

输入

一个正整数X。

输出

从1到X的所有奇数，每行一个。

范围

$1 \leq X \leq 1000$

输入

8

输出

1
3
5
7

思路 奇数序列：步长固定为2，起始值决定是奇数(1)还是偶数(2)。range(1, X+1, 2)自动跳过偶数。注意上限是X（包含），所以写X+1。

技巧 Python的range不会包含end值——这是与很多其他语言不同的地方。记住这个特征：range(a,b)实际遍历[a, b-1]。

C++

```
#include <iostream>

using namespace std;

int main()
{
    int x;
    cin >> x;

    for (int i = 1; i <= x; i += 2) cout << i << endl;

    return 0;
}
```

Python

```
x = int(input())
for i in range(1, x + 1, 2):
    print(i)
```

NQ031 正数 AcWing 712

小鲁在做班级的成绩统计："6个同学的成绩里，有几个是正数？"这需要循环+条件计数。"小华在屏幕上写了起来，"遍历每个数，如果大于0就让计数器加1。关键是理解'计数器'的概念：int cnt=0; 每满足条件就cnt++。Python更简洁——sum(1 for x in arr if x>0)。"

输入

6行，每行一个浮点数。

输出

"X positive numbers"，其中X为正数个数。

范围

 $-10^9 \leq \text{值} \leq 10^9$

输入

```
7
-5
6
-3.4
4.6
12
```

输出

4 positive numbers

思路 条件计数模式：遍历→判断→累加计数器。计数器初始化为0，每次满足条件+1。Python的sum()配合生成器表达式一行搞定。C++中浮点数与0比较注意精度。

技巧 Python生成器表达式sum(1 for x in arr if x>0)以O(n)时间和O(1)空间优雅地完成计数。C++的条件判断中整数和浮点数都适用>0。

C++

```
#include <iostream>

using namespace std;

int main()
{
    int cnt = 0;

    for (int i = 0; i < 6; i ++ )
    {
        double x;
        cin >> x;

        if (x > 0) cnt ++ ;
    }

    cout << cnt << " positive numbers" << endl;

    return 0;
}
```

Python

```
cnt = sum(1 for _ in range(6) if float(input()) > 0)
print(f"{cnt} positive numbers")
```

NQ032 连续奇数的和1 AcWing 714

"设计一个有趣的数学游戏："小华说，"随便选两个整数X和Y，然后求X和Y之间所有奇数的和。比如2和15之间的奇数是3+5+7+9+11+13+15=63。"小鲁眼睛一亮："关键是要先确定X和Y谁大谁小，再从小的遍历到大的。""没错——编程第一步总是在处理边界。"

输入

一行两个整数X和Y。

输出

一个整数，X和Y之间所有奇数的和。

范围

 $-10^6 \leq X, Y \leq 10^6$

输入

2 15

输出

63

思路 三步：1.确保起点 $\leq$ 终点（交换）；2.从起点到终点遍历每个数；3.如果是奇数则累加。Python的sum(i for i in range(lo,hi+1) if i%2==1)一行完成。C++需要手写for+if+累加。

技巧 Python的range(lo,hi+1)[lo%2::2]可以只生成奇数——利用切片+步长直接跳过偶数。但生成器表达式更可读。先确定边界再遍历是处理区间问题的通用习惯。

C++

```
#include <iostream>
#include <algorithm>

using namespace std;

int main()
{
    int x, y;
    cin >> x >> y;

    if (x > y) swap(x, y);

    int sum = 0;
    int i = x + 1;
    while (i < y)
    {
        if (i % 2) sum += i;
        i++;
    }
    cout << sum << endl;

    return 0;
}
```

Python

```
x, y = map(int, input().split())
if x > y: x, y = y, x
print(sum(i for i in range(x + 1, y)
if i % 2))
```

NQ033 最大数和它的位置 AcWing 716

小栋拿着一叠数据找到小鲁："帮我找这100个数里的最大值，还有第几个是它。"小鲁试了max()和index()："搞定了！"小华看了代码后说："不错，但你知道吗——你用了两遍遍历。更好的方法是遍历一次，同时记住最大值和位置。虽然对于100个数没区别，但如果是一亿个数，差一倍的时间就是差一个午饭。"

- 输入** 100行，每行一个整数。
- 输出** 第一行最大值，第二行位置（1-indexed）。
- 范围** $-10^9 \leq \text{值} \leq 10^9$

输入	输出
2	34565
113	4
45	
34565	
6	
...	

思路 一次遍历找最值：初设第一个为最大值位置1，然后从第二个开始，遇到更大的就更新值和位置。 $O(n)$ 一次遍历完成。这比Python的max()+index()两次遍历更高效——算法优化的意识从小题培养。

技巧 Python的max()底层也是C实现的 $O(n)$ 。但max()+index()会导致两次 $O(n)$ 遍历。手写的 $O(n)$ 单次遍历在数据量大时更优。这种"一次遍历完成多件事"的思维是算法设计的核心。

C++

```
#include <iostream>

using namespace std;
int a[100];

int main()
{
    int maxa = -1;
    int index = -1;
    for (int i = 0; i < 100; i++)
    {
        cin >> a[i];

        if (a[i] > maxa)
        {
            maxa = a[i];
            index = i;
        }
    }

    cout << maxa << endl;
    cout << index+1 << endl;

    return 0;
}
```

Python

```
n = int(input())
arr = list(map(int, input().split()))
mx = max(arr)
print(mx)
print(arr.index(mx) + 1)
```

NQ034 递增序列 AcWing 721

"有一个特殊的数据集——每个数都比前一个数大，形成一个递增序列。"小华在纸上画着："5,10,15,20,25... 我们需要一种新的循环——不知道要循环多少次，因为不知道序列有多长。""那就是while循环！"小鲁抢答，"while True读入，遇到0就break——这就是'不定长输入'的处理模式。"

- 输入** 多组，每组一个整数X。0结束。
- 输出** 对每个非0的X，输出从1到X的所有整数。
- 范围** $0 \leq X \leq 10^6$

输入	输出
5	1 2 3 4 5
10	1 2 3 4 5 6 7 8 9 10
3	1 2 3
0	

思路 while+break是不定长输入的标准处理模式：while(1){读入；if(==0) break；处理}。Python的while True和C++的while(cin>>x,x)都支持。while和for的区别在于：while适合"不知道次数"的循环。

技巧 Python: while True: x=int(input()); if x==0: break; 输出序列。C++可以用逗号表达式while(cin>>x,x)——利用逗号运算符取最后一个值。但可读性不如先读入再break。

C++

```
#include <cstdio>
#include <iostream>

using namespace std;

int main()
{
    int x;
    while (cin >> x, x)
    {
        for (int i = 1; i <= x; i++) cout << i << ' ';
        cout << endl;
    }

    return 0;
}
```

Python

```
while True:
    x = int(input())
    if x == 0: break
    print(' '.join(str(i) for i in range(1, x + 1)))
```

NQ035 连续整数相加 AcWing 720

小鲁遇到了变种题："读入A和N，求从A开始的连续N个整数的和。"比如A=3,N=2 => 3+4=7。"这跟NQ032有点像——但这次N告诉你要加几个数。"小华提示道："在循环中用一个累加器sum，循环N次，每次加上当前值然后当前值加1。Python的range(a, a+n)直接生成这一串数。"

输入 多对整数A和N，一直读到N≤0。

输出 每对输出"Sum = X"。

范围 A,N ≤ 10⁶

输入

3 2
4 -1

输出

Sum = 7

思路 连续N个整数从A+A+1+...+A+N-1 = N×A+N(N-1)/2。但循环版本更易懂：sum=0; for i in range(a, a+n): sum+=i。也可以数学公式直接算——但理解循环累加的思维更重要。

技巧 Python: sum(range(a, a+n))一行完成。但手写循环累加才能理解"累加器模式"——这是所有求和、求积、统计类问题的基础。数学公式法体现了"先思考再编码"——但不要忘了循环是理解问题的第一步。

C++

```
#include <iostream>

using namespace std;

int main()
{
    int a, n;
    cin >> a;
    while (cin >> n, n <= 0);

    int s = 0;
    for (int i = 0; i < n; i++) s += a + i;

    cout << s << endl;

    return 0;
}
```

Python

```
a = int(input())
while True:
    for n in map(int, input().split()):
        if n > 0: break
    if n > 0: break
    print(sum(a + i for i in range(n)))
```

NQ036 约数 AcWing 724

小鲁在数学课上学到了约数："6能被1,2,3,6整除——所以6有4个约数。"小华说："编程找约数很简单：从1到N遍历每个数，如果 $N \% i == 0$ 就输出。不过有个优化技巧——约数是成对出现的，你只需要遍历到 $\sqrt{N}$ 。比如6的约数：i=1找到1和6，i=2找到2和3—— $\sqrt{6} \approx 2.45$ 所以搜索到2就够了。"

输入 一个整数N。

输出 按升序输出N的所有约数，每行一个。

范围 $1 \leq N \leq 10^9$

输入	输出
6	1
	2
	3
	6

思路 基础方法：从1到N遍历， $N \% i == 0$ 则输出。优化方法：只遍历到 $\sqrt{N}$ ，每个i找到两个约数i和 N/i 。注意完全平方数（如16）时 $i = N/i$ 只算一次。 $\sqrt{N}$ 优化将复杂度从 $O(N)$ 降到 $O(\sqrt{N})$ ——对于 $N = 10^9$ ，从10亿降到3万。

技巧 C++的遍历条件 $i * i \leq N$ 比 $i \leq \sqrt{N}$ 更高效（避免浮点数）。Python用while $i * i \leq n$ 。注意输出顺序——如果要求升序，需要收集约数再排序输出。本课主要理解基本的枚举模式。

C++

```
#include <iostream>

using namespace std;

int main()
{
    int n;
    cin >> n;
    for (int i = 1; i <= n; i++)
        if (n % i == 0)
            cout << i << endl;

    return 0;
}
```

Python

```
n = int(input())
for i in range(1, n + 1):
    if n % i == 0:
        print(i)
```

NQ037 PUM AcWing 723

小鲁看到了一道来自巴西OJ的题目："输出N行，每行M个数——但每行最后一个数要替换成'PUM'。比如N=7,M=4....."小华笑道："这其实是考你对换行的精确控制——第M列不是数字而是'PUM'。关键是判断位置：如果 $(j+1)\%M==0$ 就输出PUM并换行，否则输出数字后跟空格。"

输入

一行两个整数N和M。

输出

N×M格式的序列，每行第M列输出"PUM"。

范围

 $1 \leq N, M \leq 20$

输入

7 4

输出

```
1 2 3 PUM
5 6 7 PUM
9 10 11 PUM
...
```

思路 计数器从1开始，每输出一个数+1。判断是否第M列：用 $\text{cnt}\%M==0$ （cnt从1开始）。是则输出PUM换行，否则输出数字+空格。这是格式化循环输出题——训练对循环索引和输出格式的精确控制。

技巧 Python: for i in range(n): 输出一行m-1个数字(空格分隔)+' PUM'。C++注意末尾空格的处理。理解%取模判断"是否该换行"是这类题的通用技巧。

C++

```
#include <iostream>

using namespace std;

int main()
{
    int n, m;
    cin >> n >> m;

    for (int i = 0, k = 1; i < n; i++)
    {
        for (int j = 0; j < m - 1; j++)
        {
            cout << k << ' ';
            k++;
        }

        cout << "PUM" << endl;
        k++;
    }

    return 0;
}
```

Python

```
n, m = map(int, input().split())
for i in range(1, n * m + 1):
    if i % m == 0:
        print("PUM")
    else:
        print(i, end=' ')
```

NQ038 六个奇数 AcWing 710

"给定一个整数X，输出从X开始的6个连续的奇数。"小栋出了一道题考小鲁。"但是我怎么知道X是奇数还是偶数？"小鲁问。小华提醒道："如果X是偶数，就从X+1开始——因为偶数+1就是奇数。然后用循环输出6个，每次步长2。关键是先处理奇偶性。"

输入

一个整数X。

输出

从X开始（如果X是偶数则从X+1开始）的6个连续奇数。

范围

 $-10^9 \leq X \leq 10^9$

输入

8

输出

9
11
13
15
17
19

思路 先归一化起点：if $x \% 2 == 0$: start = x + 1 else start = x。然后用for循环6次，每次输出start + 2 * i。这是"预处理起点"的标准模式——先统一格式，再统一处理。

技巧 Python一行：start = x + 1 if $x \% 2 == 0$ else x，然后for i in range(6): print(start + 2 * i)。C++可以用 $x \% 2 == 0 ? x + 1 : x$ 做起点。这种"先定点再循环"的模式在循环题中反复出现。

C++

```
#include <iostream>

using namespace std;

int main()
{
    int x;
    cin >> x;

    if (x % 2 == 0) x ++ ;

    for (int i = 0; i < 6; i ++ ) cout << x + i * 2 << endl;

    return 0;
}
```

Python

```
x = int(input())
if x % 2 == 0:
    x += 1
for _ in range(6):
    print(x)
    x += 2
```

NQ039 乘法表 AcWing 711

"九九乘法表——这是我奶奶都会背的东西！"小鲁笑道。"但用代码生成它，需要你在一个循环里再套一个循环——这就是嵌套循环。"小华打开TRAE演示，"外层控制行，内层控制列。第*i*行第*j*列输出*i*j*。两层for循环，一共*N*×*N*次操作。注意格式对齐——每个数占4个字符宽度。"

输入

一个整数*N*（表示表格大小）。

输出

N×*N*的乘法表，每个数占4宽对齐。

范围

$1 \leq N \leq 15$

输入

3

输出

```
1  2  3
2  4  6
3  6  9
```

思路 嵌套循环的标准结构：for *i* in range(1,*N*+1): for *j* in range(1,*N*+1): print(*i*j*)。内层循环完整执行*N*次后外层才进入下一次。注意格式化宽度——Python的f'{*i*j*:4d}'。嵌套循环是程序设计的第一个心智跃迁。

技巧 Python的f'{val:4d}'表示4宽度右对齐。C++用setw(4)或printf("%4d")。嵌套循环复杂度 $O(N^2)$ ——当*N*翻倍时运算量翻4倍。这是理解"时间复杂度"的入门。

C++

```
#include <stdio>

int main()
{
    int n;
    scanf("%d", &n);

    for (int i = 1; i <= 10; i ++ ) printf("%d x %d = %d\n", i, n, i * n);

    return 0;
}
```

Python

```
n = int(input())
for i in range(1, 11):
    print(f"{i} x {n} = {i * n}")
```

NQ040 实验 AcWing 718

小鲁的生物课在做动物实验：统计N组数据中青蛙、老鼠和兔子的总数及占比。"每组数据有两个数：数量和动物类型——C代表rabbit、R代表rat、S代表frog。""这不就是分类统计吗？"小华说，"用三个计数器分别统计，最后算百分比。Python可以用collections.Counter自动分类，但手写三个if分支更直观。"

输入 第一行N。接下来N行，每行"数量 类型(C/R/S)"。

输出 实验动物总数+各类数量+百分比。

范围 $1 \leq N \leq 100$

输入

3
5 C
7 R
8 S

输出

Total: 20 animais
Total de coelhos: 5
Total de ratos: 7
Total de sapos: 8
Percentual de coelhos: 25.00 %
Percentual de ratos: 35.00 %
Percentual de sapos: 40.00 %

思路 分类统计：three counters。每次根据类型字符判断加到哪个计数器。最后算百分比=各类/总数×100。注意保留两位小数。Python的if-elif或dict映射都比C++的if-else更简洁。

技巧 Python用dict映射：cnt={'C':0,'R':0,'S':0}，然后cnt[t]+=qty。比三个if更灵活。百分比输出用f"{cnt['C']/total*100:.2f} %"。注意%号的转义——f-string中用%%。

C++

```
#include <cstdio>
#include <iostream>

using namespace std;

int main()
{
    int n;
    cin >> n;

    int c = 0, r = 0, f = 0;
    for (int i = 0; i < n; i++)
    {
        int k;
        char t;
        cin >> k >> t;
        if (t == 'C') c += k;
        else if (t == 'R') r += k;
        else f += k;
    }

    int s = c + r + f;
    printf("Total: %d animals\n", s);
    printf("Total coneys: %d\n", c);
    printf("Total rats: %d\n", r);
    printf("Total frogs: %d\n", f);
    printf("Percentage of coneys: %.2lf %%\n", (double)c /
s * 100);
    printf("Percentage of rats: %.2lf %%\n", (double)r / s
* 100);
    printf("Percentage of frogs: %.2lf %%\n", (double)f / s
* 100);

    return 0;
}
```

Python

```
n = int(input())
c = r = f = 0
for _ in range(n):
    k, t = input().split()
    k = int(k)
    if t == 'C': c += k
    elif t == 'R': r += k
    else: f += k
s = c + r + f
print(f"Total: {s} animals")
for name, cnt in [('coney', c), ('rat', r), ('frog', f)]:
    print(f"Total {name}: {cnt}")
for name, cnt in [('coney', c), ('rat', r), ('frog', f)]:
    print(f"Percentage of {name}: {cnt/s*100:.2f} %")
```

NQ041 余数 AcWing 715

小鲁在整理数据时发现了一个规律："这组数里，除了2以外，其他被2整除的都不对……"小华纠正道："你说的是'除了2以外的所有整数，除以2的余数都是0或1'对吧？这道题就是数一数从1到10000有多少个数除以N余2。'小小的循环条件就能精准筛选。"

输入

一个整数N。

输出

从1到10000中除以N余2的所有数，每行一个。

范围

 $1 \leq N \leq 10000$

输入

13

输出

2
15
28
...

思路 条件循环：for i in range(1,10001): if i%N==2: print(i)。取模运算%是循环中最高频的条件运算符——余数判断可用于奇偶(i%2)、整除(i%N==0)、特定余数(i%N==k)等。

技巧 if i%N==2表示i除以N的余数等于2。比如N=13时，2÷13=0余2，15÷13=1余2，28÷13=2余2……余数是最基础的数论概念之一——理解它，后续的哈希、质数等问题才不会被拦在门外。

C++

```
#include <iostream>

using namespace std;

int main()
{
    int n;
    cin >> n;

    for (int i = 1; i < 10000; i ++ )
        if (i % n == 2)
            cout << i << endl;

    return 0;
}
```

Python

```
n = int(input())
for i in range(1, 10001):
    if i % n == 2:
        print(i)
```

NQ042 区间2 AcWing 713

"上次我们数了正数有多少个——现在升级一下，统计位于[10,20]区间内的个数，以及区间外的个数。"小栋把题目升级了。小鲁自信满满："这简单！读入N个数，如果x在10到20之间（包含两端），in_counter++；否则out_counter++。一道题，两个计数器，一次遍历。"

输入 第一行N。接下来N个整数。

输出 "X in"换行"Y out"。

范围 $1 \leq N \leq 10000$

输入	输出
4	2 in
14	2 out
123	
10	
-25	

思路 双计数器+区间判断：in_cnt统计 $10 \leq x \leq 20$ ，out_cnt统计x其他。一次遍历完成两个统计任务。Python用链式比较 $10 \leq x \leq 20$ 最直观。注意包含端点——10和20都算in。

技巧 Python链式比较：if 10 <= x <= 20。比C++的 $x >= 10 \ \&\& \ x \leq 20$ 更自然。两个计数器互斥且完备——in+out = N，可以只统一个数推另一个，但分别统计更安全（验证用的N个都处理了）。

C++

```
#include <iostream>

using namespace std;

int main()
{
    int n;
    cin >> n;

    int x = 0, y = 0;
    while (n -- )
    {
        int t;
        cin >> t;
        if (t >= 10 && t <= 20) x ++ ;
        else y ++ ;
    }

    cout << x << " in" << endl;
    cout << y << " out" << endl;

    return 0;
}
```

Python

```
n = int(input())
in_cnt = sum(1 for _ in range(n) if 1
0 <= int(input()) <= 20)
print(f"{in_cnt} in")
print(f"{n - in_cnt} out")
```

本章知识点总结

- **for循环**：确定次数的循环首选。Python的range(start,end,step)注意end不包含
- **while循环**：不确定次数的循环首选。while+break处理不定长输入
- **嵌套循环**：外层行内层列，复杂度 $O(N^2)$ 。理解两次for的执行顺序
- **条件循环**：循环中嵌入if判断——计数、筛选、分类，90%的算法代码都是这个模式
- **累加器模式**：sum/cnt初始化为0，循环中逐次更新。这是所有统计/求和的基础
- **先思考再编码**：约数的 $\sqrt{N}$ 优化、等差数列求和公式——数学知识直接减少循环次数